# SINAIS, PROCESSOS FILHOS E EXECUÇÃO EM PARALELO

*O operador apertou **Ctrl-C**. O script parou de imprimir e o prompt voltou. **Trinta segundos depois, o roteador do laboratório caiu.** O script havia lançado doze processos com `&`; o `SIGINT` foi ao grupo do primeiro plano e o shell morreu, mas alguns filhos já haviam trocado de grupo e outros ignoravam `SIGINT`. O tráfego continuou saindo por meia hora até alguém rodar `pkill`. **Um script que cria processos assume a responsabilidade por eles** — "parece que parou" não é "parou".*

**075**
GUIAS LINUX

## CICLO DE VIDA DO FILHO · OS SINAIS QUE IMPORTAM · O TETO DE CONCORRÊNCIA

**PAI LANÇA** comando &

**FILHO EXECUTANDO** $! guarda o PID

**FILHO TERMINOU** devolveu um status

**PAI FEZ WAIT** status colhido

pai morreu ↓

**ÓRFÃO** continua rodando, adotado pelo `init` — **FORA DO ALCANCE DO SEU CTRL-C**

**ZUMBI** terminou, mas ninguém leu o status: ocupa entrada na tabela de processos

↑ sem wait

Num laço que lança milhares de filhos sem `wait`, os zumbis **esgotam o limite de processos do usuário**. As duas curas são a mesma disciplina: **todo filho lançado tem um `wait` correspondente**, e todo script que lança filhos tem um `trap` que os mata antes de sair.

### SINAIS EM SCRIPT · INTERCEPTÁVEL?

| | | |
|---|---|---|
| `SIGINT` | 2 | sim — `Ctrl-C` no terminal |
| `SIGTERM` | 15 | sim — **o pedido educado**: `kill` sem argumento, `systemctl stop` |
| `SIGKILL` | 9 | **NÃO** — nenhum trap roda, nenhuma limpeza acontece |
| `SIGHUP` | 1 | sim — terminal fechado; recarga de configuração |
| `SIGPIPE` | 13 | sim — escreveu num pipe sem leitor |
| `SIGQUIT` | 3 | sim — `Ctrl-\` |
| `SIGCHLD` | 17 | sim — um filho terminou (padrão: ignora) |

**Morte por sinal devolve `128 + N`: 130** é Ctrl-C, **143** é `SIGTERM`, **137** é `SIGKILL`. Reconhecer esses números no log poupa investigação.

`for …; do cmd & done` — p1 p2 p3 p4 p5 p6 p7 p8 p9 p10 p11 p12 — 12 processos disputando 4 núcleos e uma placa de rede

`xargs -P 4 · wait -n` — p1 p2 p3 p4 ◀ p5 p6 p7 p8 p9 p10 p11 p12 — 4 executando · 8 na fila, entram quando um sai

**O teto certo vem do gargalo, não da sua vontade.** Trabalho de **CPU**: perto do número de núcleos (`nproc`). Trabalho de **disco** em HD mecânico: 1 ou 2 — mais que isso faz o cabeçote passar o tempo se deslocando, e fica **mais lento que sequencial**. Trabalho de **rede**: o número combinado com quem opera o alvo, e não o que a sua máquina aguenta. E a etapa final de agregação é **sempre sequencial** — é o gargalo que sobra depois de paralelizar tudo o que dava. As três formas de impor o teto: `xargs -P 4 -n 1` (nada a instalar, mas **intercala a saída** — cada processo escreve no próprio arquivo, a junção vem no fim); `parallel -j 4 --halt now,fail=1 --joblog t.log` (serializa a saída, registra, retoma o que falhou, para no primeiro erro — custa uma dependência); e a fila em Bash puro, `while (( $(jobs -rp | wc -l) ≥ MAX )); do wait -n; done`, que não depende de nada além do Bash 5.1.

## LANÇAR, COLHER O STATUS, E MATAR JUNTO

kali@kali: ~/lab-paralelo

```
# o esqueleto de qualquer paralelismo em Bash: PID → unidade
$ declare -A pid_de
$ for alvo in "${alvos[@]}"; do
    processar "$alvo" > "saida-$alvo.txt" &
    pid_de[$!]="$alvo"   # $! muda a cada &: capture já
  done
$ for pid in "${!pid_de[@]}"; do
    wait "$pid" || { echo "falhou: ${pid_de[$pid]}" >&2; ((falhas++)); }
  done
# o trap que faz o Ctrl-C parar DE VERDADE
$ encerrar() {
    trap - INT TERM      # evita reentrar no 2º Ctrl-C
    kill -TERM "${filhos[@]}" 2>/dev/null || true
    sleep 2
    kill -KILL "${filhos[@]}" 2>/dev/null || true
    wait 2>/dev/null; exit 130
  }
$ trap encerrar INT TERM
# prazo: 124 = estourou; -k mata quem ignora o TERM
$ timeout -k 5 30 varrer "$alvo"; echo $?
124
$ pgrep -af 'varrer' || echo "nenhum orfao -- correto"
nenhum orfao -- correto
```

### O TRAP QUE NÃO DISPARA

Se o script espera um **comando externo em primeiro plano**, o trap **fica pendente até esse comando retornar**: um script parado em `sleep 300` que recebe `SIGTERM` só executa o handler depois dos 300 segundos. A cura é pôr o comando longo em segundo plano e usar `wait`, que **é** interrompível — `sleep 300 & filho=$!; wait "$filho"`. É exatamente a diferença entre um serviço que para no `systemctl stop` e um que estoura o `TimeoutStopSec` e leva `SIGKILL`.

### MATAR OS FILHOS: TRÊS FORMAS

| | |
|---|---|
| `filhos=()` | **registre os PIDs e mate no trap** — previsível, resolve a esmagadora maioria dos casos. É a que se deve preferir |
| `kill -TERM -$$` | PID **negativo** = o grupo inteiro. Só funciona se o script for líder do próprio grupo; chamado de dentro de outro script, **mata o chamador junto**. Confira com `ps -o pid,pgid,comm -p $$` |
| `setsid --wait` | lança num grupo novo e mata só esse grupo — isolamento garantido, ao custo de uma camada a mais |
| `TERM → KILL` | nunca **-9** de primeira: com `SIGKILL` o filho **não remove os próprios temporários** nem fecha os arquivos |

### SOBREVIVER AO TERMINAL

| | |
|---|---|
| `nohup` | imune ao `SIGHUP` desde o lançamento |
| `disown -h` | protege um job **já lançado** |
| `setsid` | sessão própria, desligado do terminal |
| `systemd` | **a resposta certa para trabalho longo:** registro no `journald`, reinício automático e `KillMode=control-group`, que mata o cgroup inteiro — a garantia que o script sozinho não tem. `tmux` quando for preciso **voltar a interagir** |

### A ARMADILHA DO >( ) COM WAIT

O shell **não considera** o processo de `>(…)` um filho a esperar. Em `gerar | tee >(lento > r.txt) >/dev/null`, o `r.txt` pode estar vazio na linha seguinte. Quando a ordem importa, use PID explícito: `lento < <(gerar) > r.txt & pid=$!; wait "$pid"`.

**Um script que lança filhos e não tem `trap INT TERM` continua trabalhando depois que o operador mandou parar.** É o pior caso deste assunto: a tela para, o prompt volta, e o tráfego continua saindo — foi assim que o roteador da abertura caiu meia hora depois do `Ctrl-C`. Registre os PIDs ( `filhos+=("$!")` ), mate com `TERM`, espere, e só então `KILL` no que sobrou. **Cuidado redobrado com `kill -TERM -$$`** (PID negativo = grupo inteiro): se o script não for líder do próprio grupo, ele mata **o shell que o chamou e tudo o que estiver junto** — confirme com `ps -o pid,pgid,comm -p $$` antes, e prefira a lista explícita de PIDs. E **nunca use `pkill -f` com padrão curto** para limpar a bagunça: `pkill -f python` derruba serviços que nada tinham a ver. Ensaie o encerramento com `echo kill` no lugar do `kill`, em VM descartável, antes de confiar no handler.

## RESUMO RÁPIDO

- ✓ `SIGKILL` **não pode ser interceptado** — nenhum trap roda, nenhuma limpeza acontece. Sempre `TERM`, pausa, e `KILL` só no que sobrou.
- ✓ **Morte por sinal devolve `128 + N`:** 130 é Ctrl-C, 143 é `SIGTERM`, 137 é `SIGKILL`.
- ✓ **Trap não dispara enquanto um comando externo está em primeiro plano.** Ponha o comando longo em segundo plano e use `wait`, que é interrompível.
- ✓ **Capture `$!` na linha seguinte ao `&`**, porque ele muda a cada lançamento. Um array associativo de PID para unidade de trabalho é o esqueleto de qualquer paralelismo em Bash.
- ✓ `wait` **sozinho não revela qual filho falhou** — colha os status um a um se o resultado importar. Num relatório de cobertura, sempre importa.
- ✓ **Todo script que lança filhos precisa de um `trap INT TERM` que os mate.** Sem isso, o Ctrl-C para a tela e não para o trabalho.
- ✓ **Imponha teto de concorrência** com `xargs -P`, `parallel -j` ou fila com `wait -n`. O número vem do gargalo: núcleos para CPU, 1 ou 2 para disco mecânico, o combinado para rede.
- ✓ `timeout -k` **em toda ferramenta externa.** Sem `-k`, quem ignora o `TERM` sobrevive ao prazo; o status **124** identifica o estouro.
- ✓ **Para trabalho longo de verdade, uma unidade do systemd vence o `nohup`:** registro, reinício e encerramento do cgroup inteiro.

## NO KALI

**Paralelismo contra alvo externo é decisão de contrato, não de máquina.** `-P 8` multiplica por oito o tráfego que você gera: o teto vem do que foi combinado com quem opera o alvo — taxa máxima, janela de execução, limite de conexões simultâneas. Estourá-lo por acidente é **indisponibilidade provocada**, e a autorização para testar não cobre negar o serviço. `parallel --delay 0.5` e `--jobs` explícito documentam a intenção no próprio comando. **O Ctrl-C tem de parar de verdade** — é o ponto mais importante deste assunto em contexto ofensivo: sem o `trap` que mata os filhos, o script "para" na tela e **continua atacando por baixo**, o que num teste com janela combinada coloca a atividade **fora da janela autorizada**, com o enquadramento da Lei 12.737/2012. `timeout -k` **em toda ferramenta externa:** varredura contra host que descarta pacotes fica pendurada indefinidamente, e uma fila de oito processos com quatro travados não anda — prazo explícito, `-k` para o caso de a ferramenta ignorar o `TERM`, e status 124 tratado como "sem resposta", não como "erro". **Registre PID e alvo, e confira o status um a um:** o `pid_de[$!]="$alvo"` é o que permite dizer no relatório quais alvos foram efetivamente testados e quais falharam — um `wait` sozinho devolve zero e produz relatório que **afirma cobertura que não houve**, o pior tipo de erro num teste de segurança. E **interrupção deixa evidência parcial:** ao encerrar por sinal, marque no arquivo de resultado que a execução foi interrompida e em que ponto — arquivo truncado sem marca parece completo, e alguém vai concluir que os alvos restantes estavam limpos. **Família RHEL:** sinais e controle de processos são do kernel e idênticos; a diferença é de operação — `KillMode=control-group` nas unidades do systemd mata todos os processos do cgroup no `systemctl stop`, que é justamente a garantia que o script sozinho não tem, mais um argumento para empacotar trabalho longo como serviço. E o `parallel` exige EPEL no RHEL, enquanto `xargs -P` e `timeout` estão nas duas famílias sem instalar nada.